Martin-Luther-Universität Halle-Wittenberg
Faculty of Natural Science II
Institute of Physics

Physics Modelling Assistant

Using Agentic Artificial Intelligence in Physics Modelling

Bachelor's Thesis
for the Award of the Academic Degree
Bachelor of Science
in Physik und Digitale Technologien

submitted by: **Janis Felix Jacobi**
Supervisor 1: Prof. Dr. Niels Schröter
Supervisor 2: Prof. Dr. Samir Lounis

submission: Halle (Saale), 24th August 2026

Contents

Figure Index

Table Index

Glossary

Agent a singular AI system which is commonly part of an Agentic AI system 3, 4, 10–19, 21

Agentic AI an AI system with enhanced capabilities (subsubsection 2.1.3) 2, 4, 5, 10–12

AI Artificial Intelligence 2, 4–6, 9–13

API key application programming interface key; a unique and secret identifier granting access to a service 4, 13, 14, 20

context-window a size describing how many previous output tokens a LLM can take into consideration 4, 6, 11, 16–18

LLM Large Language Model 2, 4–19, 21

logit a LLM's output for every token indicating how well it is suited to be the next token. Larger values indicate a better fit. [31, p. 4] 4, 7, 8

token the input for a LLM's transformation function 4, 6–11, 13, 16–18

1 Introduction

Artificial Intelligence can be of great assistance in scientific research [47]. As shown by Chugunova et al. [7], researchers in natural science are among the groups most familiar with AI compared to other scientist. There is a wide range of applications stretching from finding sources in the literature to aiding in the presenting research findings. It can help in forming hypothesis and in evaluating them by designing and controlling practical experiments and aiding during simulations, especially with the program implementation.[47]

In the past two years automated Agentic AI research pipelines where developed with the potential to improve the work wiht AI in science. The first fully autonomous approach was published by Lu et al. in 2024[38], introducing a framework called "The AI Scientist"[38]. It identifies interesting research questions, plans and executes according experiments, interprets the results and writes a paper stating its findings via a multistep process. A similar system is the "InternAgent"[44], which provide an interface for introducing human feedback into the system. This is realised by the "Orchestration Agent"[44] managing how and when human input is introduced into the system. The system itself consists of the four distinct tasks of "Survey", "Code Review", "Assessment" and "Idea Innovation"[44]. The "Orchestration Agent"[44] manages the interactions between the tasks and how the findings are presented to the human. This system shows some resemblance to human research groups with individuals specialized in specific aspects of the work while a supervisor coordinates the process. In tests ran by the "InternAgent" developers Team et al., their system was able to outperform the "AI-Scientist-V2" [50], a successor of "The AI Scientist"[38].

The work on the "InternAgent" [44] also introduced tracking the cost of the AI usage during a run of the pipeline. In this metric the "InternAgent" [44] outperformed the "AI-Scientist-V2"[50] again, indicating that higher costs do not necessarily yield better results.

A less automated approach was made by Shao et al. with the introduction of "SciSciGPT" [42]. This system is for Science of Science research, which is why "SciSciGPT" is equipped with data management component amongst a literature review and an analytics component. Similar to the "InterAgent" some form of orchestrator is used. It receives the outputs of the subtasks along with an evaluation from a task designated to critique the outputs of other tasks. In contrast to the "AIScientist" [38] and "InternAgent" [44], "SciSciGPT" [42] does not generate its own research ideas. Instead it responds to an input prompt, allowing for seamless integration into the work of human scientists.

As of 1st June 2026 the latest advancement in the field was made by Louapre, Niklaus, and Tunstall presenting the "physics-intern"[37], "a multi-agent scaffolding system that takes a physics problem and works through it autonomously"[37]. Their approach is similar to previous systems, using an orchestrator to manage a research and computation component. The results are reviewed and checked against the literature before the critique is passed to the "Strategy Planner"[37], which has access to a literature "Background Survey"[37] and can adjust the orchestration strategy.

This Thesis aims to develop a system with functionality similar to the "physics-intern"[37] (at the time the work on this Thesis began, the "physics-intern"[37] was not yet published). The Physics Modelling Assistant, receives a physics problem or modelling request and crafts an according model and its implementation to answer the request. The performance of the system will be measured to provide inside into its usefulness. The influence of some of the system's configurations will be explored by measuring their effect on the performance. With this work I want to contribute to a better understanding of LLM's parameters and a more efficient use of AI systems in scientific research.

2 Theoretical Foundations

Before going in to the details of AI-based systems, some key terms will be defined.

2.1 Definitions

2.1.1 Artificial Intelligence

Artificial Intelligence (AI) describes the functionality of a system. There is no universally accepted definition for AI, so for the purpose of this Thesis, the definition from Russell and Norvig [41, p. 22-26] will be used. They describe AI as being a rational actor (rational agent), which acts in a way that achieves the best result based on the expectation of what the result should be ([41, p, 26], translated). This definition allows for a variety of systems to be considered an AI and accounts for the uncertainty regarding the goal the system should work towards. Classical PID-controllers and software for processing and displaying data, as crafted by Jacobi [30], therefore could be considered AI systems.

2.1.2 Large Language Model

A Language Model at it's core is a "Transformer Language Model"[52] as described by Zhao et al. It operates on small words or parts of words, which are called tokens.

When a LLM receives an input string, it will divide it into tokens and put into the transformation function. Based on these input tokens and the previous output tokens the model calculates the most likely next output tokens. The number of tokens a model can see and therefore consider is limited, the limit is called the context-window of the model [34]. The result of the LLM is its calculations for most likely sequence of output tokens given the provided input, which means, it provides the best result for the given goal, thereby fulfilling the definition of AI (see subsubsection 2.1.1). The process of developing capable LLMs will not be discussed in this Bachelor's Thesis.

The distinction between Language Models and Large Language Models is in the number of internal parameters a model has. As stated in *A Survey of Large Language Models*[52], there is no hard cut-off for a Language Models to be considered Large. Commonly, models which are considered Large Language Models have $\geq 10 \cdot 10^9$ parameters.

2.1.3 Agentic AI

As described by Bandi et al., Agentic AIs are "autonomous, goal-driven systems that can operate on their own for long periods, requiring minimal human supervision"[5, section:3. Results]. In order to achieve this behaviour, these systems combine abilities which are beyond LLMs. Bandi et al. highlight the "strategic planning" [5, section:3. Results] capabilities, the preservation of the tasks context over time [5], the usage of "external tools to extend [the systems] capabilities" [5] and the collaboration of individual Agents (this means AI-systems) with other Agents[5].

2.2 Decoding the Output of LLMs

In subsubsection 2.1.2 LLM is defined based on the functionality it provides. In order to understand how the behaviour of LLMs can be modified, a more formal definition will be given, based on the work of Ji et al. in *Decoding as Optimisation on the Probability Simplex: From Top-K to Top-P (Nucleus) to Best-of-K Samplers* [31].

2.2.1 LLM as a Function which Assigns Probabilities

A LLM can be described as a function f with internal parameters θ. Additionally, f takes a set of input token $t_{in} = (i_1, ..., i_N)$, where $N \in \mathbb{N}$ is the number of input tokens, and a set of previous output tokens $v_{out,prev} = (o_{-1}, ..., o_{-M})$, where $M \in \mathbb{N}$ is the number of previous output tokens considered. The function $f(\theta, t_{in}, t_{outp,prev})$ assigns a probability $s(v)$ to every token $v \in \mathcal{V}$, where $\mathcal{V}$ is the vocabulary of tokens known to the LLM:

$$f(\theta, t_{in}, t_{outp,prev}) = \begin{pmatrix} s(v'_1) \\ ... \\ s(v'_Z) \end{pmatrix} = s(v) \quad \text{where } Z \text{ is the size of the vocabulary } \mathcal{V} \tag{2.2.1}$$

v'_i with $i \in [1, Z] \subset \mathbb{N}$ is an element from the ordered set of $\mathcal{V}'$, which allows to map the rows of the probability matrix $s(v)$ to the tokens. The probability indicates how well a token is suited to be the next output token[31, p. 4]. The probabilities $s(v'_i)$ are called "logits"[31, p. 4], they are not normalized by default and larger values of a logit signal a better fit for the corresponding token to be the next output token.

The LLM by itself outputs a probability distribution including every token from its vocabulary. To choose a token based on this probability distribution, Ji et al. state that the following optimization problem needs to be solved, they label it the "MASTER PROBLEM"[31, p. 5]:

$$q^* = \arg\max_{q \in \Delta(\mathcal{V})} \left[\underbrace{\langle q, s \rangle}_{\sum_{v \in \mathcal{V}} q(v) \cdot s(v)} - \lambda \Omega(q) \right] \tag{2.2.2}$$

The used quantities are:

- q^* is the probability distribution maximizing Equation 2.2.2
- $\Delta(\mathcal{V})$ is the set of all possible probability distributions
- q is a single probability distribution distributions for a given vocabulary $\mathcal{V}$
- s is the initial vector of logits the LLM produces
- $\Omega(q)$ is a regularisation function, which can push q^* towards a certain distribution
- λ is the strength of the regularisation function and $\lambda \geq 0$

There are different approaches for finding q^* and eventually getting a single token from it, the once relevant to this Thesis will be discussed in the following.

2.2.2 Greedy Decoding [31, p. 10]

This method uses no special regularisation function $\Omega(q)$, and therefore sets sets $\lambda = 0$. This simplifies Equation 2.2.2 to $q^* = \arg\max_{q \in \Delta(\mathcal{V})} \langle q, s \rangle$. The resulting q^* vector has all the probability mass placed on the token, which was calculated to be most likely, while all other tokens get assigned a probability of 0. Therefore, the vector $q*$ has value 0 in every row except the one corresponding to the most likely token, which has probability 1. This means the next token will always be the most likely token according to the LLM's calculations. If multiple tokens have the highest probability $s(v)$, a single token is selected [31, p. 10].

2.2.3 Softmax Decoding

For Softmax Decoding, the regularisation function is chosen as $\Omega(q) = -\sum_{v \in \mathcal{V}} q(v) \log(q(v))$, which is the negative entropy of q. Using this $\Omega(q)$ in solving Equation 2.2.2 leads to the following optimal distribution q^*, where $q^*(v)$ is the probability per token[31, p. 10-12]:

$$q^*(v) = \frac{e^{\frac{s(v)}{\lambda}}}{\sum_{u \in \mathcal{V}} e^{\frac{s(u)}{\lambda}}} \tag{2.2.3}$$

This distribution is normalised and has the parameter λ, which controls how the distribution q^* is spread. For larger values of λ, the probability is spread more evenly across all tokens which results in previously unlikely tokens becoming more likely. When λ becomes very small, the probability distribution is sharper, with the probability mass being concentrated around a few tokens which already where likely. LLM calculates to me suitable become more likely. The next output token is selected according to the final probability distribution. For the edge case $\lambda = 0$, Softmax Decoding becomes Greedy Decoding (see subsubsection 2.2.2).

The parameter λ is one way of controlling the general randomness of the output. In the context of LLMs this parameter is often associated with the creativity of a model and called **Temperature** T [31, p. 10-12].

2.2.4 Top-k Decoding

Top-k Decoding extends Softmax Decoding (subsubsection 2.2.3) by selecting the k ($k \in \mathbb{N} \setminus k = 0$) tokens with the highest logit values [31, p. 12]. These tokens form a subset of the vocabulary called $\mathcal{V}_k$ on which Softmax Decoding is performed, while the probability for every other token $v \in \mathcal{V} \setminus \mathcal{V}_k$ is set to 0. This gives the following probability distribution[31, p. 12]:

$$q^*(v) = \begin{cases} \frac{e^{\frac{s(v)}{\lambda}}}{\sum_{u \in \mathcal{V}_k} e^{\frac{s(u)}{\lambda}}} & \text{for } v \in \mathcal{V}_k \\ 0 & \text{else} \end{cases} \tag{2.2.4}$$

For $k \geq |\mathcal{V}|$, Top-k Decoding is equivalent to the standard Softmax Decoding (see subsubsection 2.2.3). Setting $k = 1$ results in the same functionality as Greedy Decoding (see subsubsection 2.2.2).

Top-k Decoding provides another parameter called **top_k**, which can be used to shape the output of LLMs [31, p. 12].

2.2.5 Top-p Decoding

Top-p Decoding is another extension to Softmax Decoding (subsubsection 2.2.3), also selecting a subset of all tokens called $\mathcal{V}_p$ from the vocabulary $\mathcal{V}$. In contrast to Top-k Decoding (subsubsection 2.2.4), Top-p Decoding defines a certain amount of probability $p \in (\, 0, 1\,]$, which will be considered for the Softmax Decoding. This is achieved by selecting the token with the highest probability $s(v)$ from the set $\mathcal{V} \setminus \mathcal{V}_p$ and add it to the set $\mathcal{V}_p$, thereby removing it from $\mathcal{V} \setminus \mathcal{V}_p$. This is repeated until the sum of the probabilities of all tokens $v \in \mathcal{V}_p$ exceeds the value of p. The probability of the tokens in $\mathcal{V} \setminus \mathcal{V}_p$ is set to 0, while Softmax Decoding (subsubsection 2.2.3) will be performed on the tokens in $\mathcal{V}_p$. The resulting probability distribution is (from [31]):

$$q^*(v) = \begin{cases} \frac{e^{\frac{s(v)}{\lambda}}}{\sum_{u \in \mathcal{V}_p} e^{\frac{s(u)}{\lambda}}} & \text{for } v \in \mathcal{V}_p \\ 0 & \text{else} \end{cases} \tag{2.2.5}$$

For $p = 1$, Top-p Decoding is equivalent to Softmax Decoding, since all tokens will be included . For $p \to 0$, only the token with the highest probability gets a non 0 probability, which makes this case equivalent to Greedy Decoding (see subsubsection 2.2.2).

Top-p Decoding allows for dynamic adaptation based on how sharp or spread out the probability distribution given by the LLM is. For a sharp distribution, only the few very likely tokens keep their calculated probabilities and Softmax Decoding (subsubsection 2.2.3) will be performed on them. Whereas, for a flatter distribution, a higher number of tokens will be included into the Softmax Decoding (subsubsection 2.2.3) and therefore are possible output tokens.

The derived parameter is called **top_p** and is commonly used in the field of LLMs to modify a system's output [31, p. 13].

2.3 Other Parameters which Can Influence an AI System's Performance

As shown by numerous benchmarks ([26], [27]) different LLMs achieve different performances. Some of th important factors for these differences will be discussed in the following.

2.3.1 Selection of the LLM

One of the most noticeable differences in LLM systems is the number of internal parameters used. The general assumption is that more parameters lead to better performance. Another important feature is the amount of data used to train a LLM. As shown by Kaplan et al.([32, p. 5]), combining the number of internal parameters N a model uses and the number of tokens D used in training a model can predict the "cross-entropy loss" [32, p. 6] L of a LLM:

$$L(N, D) = \left[\left(\frac{N_C}{N} \right)^{\frac{\alpha_N}{\alpha_D}} + \frac{D_C}{D} \right]^{\alpha_D} \tag{2.3.1}$$

The used quantities are:

- $L(N, D)$ is the "cross-entropy loss"[32, p. 6]. Lower Values indicate a better model.

- N_C is the critical number of internal parameter until which Equation 2.3.1 is applicable. $N_C \approx 8.8 \cdot 10^{13}$ [32]
- N is the number of internal parameters the model uses.
- D_C is the critical number of tokens until which Equation 2.3.1 is applicable. $D_C \approx 5.4 \cdot 10^{13}$ [32]
- α_N is the exponent for predicting $L(N)$. $\alpha_N \approx 0.076$ [32]
- α_D is the exponent for predicting $L(D)$. $\alpha_D \approx 0.095$ [32]

There are also other design and training related influences on a models performance, Kaplan et al. discus the relevance of the number of training steps or the batch size (tokens per step) [32]. Liu et al. [36] show the importance of how the training data is composed, highlighting the increases in predicting a models performance when considering this factor.

2.3.2 Tool Use of LLMs

Since all LLMs have a limited amount of training data, there are domains a model is lacking basic knowledge in, hindering it to perform tasks in this domain effectively. In order to mitigate this, LLMs are fed extra information to help them answer the question at hand. Realizing this via human input is inefficient and impractical, especially when the human itself is not very knowledgeable in the domain at hand. For these reasons, LLMs are getting equipped with tools, allowing them to access information, which is not included in their training data. As shown by Komeili, Shuster, and Weston [33] and Thoppilan et al. [46], equipping a LLM with some sort of web search tool allows to reduces the false outputs a system produces. Note that tool use is one of the features making an AI an Agentic AI, as discussed in subsubsection 2.1.3.

Another common class of tools are file readers, they can be able to parse various formats like PDF, JSON, or plain text.

Code execution tools allow a LLM run generated code autonomously. This introduces the problem of untrusted code execution, since the model could generate malicious code, accessing private information or modifying important data on the system it executes the code on. Executing untrusted code should therefore be done in a container separate from important data and systems. As stated by Marchand et al. [39], these containers, called Sandboxes, are not always fully secure; strong LLMs are able to escape them sometimes.

2.4 Orchestrating AIs to Behave as an Agentic AI

As mentioned in subsubsection 2.1.3, combining multiple AI systems into one enables the resulting system to perform more complex tasks and achieve better results. The individual AI systems of an Agentic AI system are commonly referred to as Agents. In the following, the common approaches on how Agents can work together will be presented, as there where outlined by Kulkarni and Kulkarni [35].

2.4.1 Orchestration Architectures

The Sequential Pipeline [35, p. 3] is a linear chain, where each Agent is executed one after another [35]. Individual Agents or their tools can be rerun, as long as the pipeline is still in the corresponding step. But

once an Agent is done, it can not be called again (in the same run). The design process is relatively simple and the resulting system show very deterministic behaviour, which allows errors to be located easily [35].

The Parallel Fan-Out with Merge [35, p. 4] differs from the Sequential Pipeline (paragraph 2.4.1), by allowing certain Agents to run simultaneously. After the parallel Agents are done with their tasks, their results are merged by the next sequential Agent. Introducing an Agent before the parallelization allows for each parallel Agent only receive the information relevant to its task. Therefore, each parallel Agent has to deal with less noise and receives less input tokens, which helps to stay within the LLM's context-window. The Agent responsible for merging resolves conflicts like contradicting information of parallel Agents. The parallel design allows for the highest throughput compared to the other architectures [35, p. 7].

The Hierarchical Supervisor-Worker [35, p. 4-5] architecture introduces a supervisor Agent. It dynamically decides which Agents (Subagents) to call and which tasks the Subagents should handle. Furthermore, the supervisor Agent reviews the Subagent's results and reruns the respective tasks, if it has low confidence in the result being correct. This forms a feedback loop, which allows the Agentic AI system to dynamically assign more difficult tasks to more powerful Agents. The hierarchical architecture produces better results than the Sequential Pipeline (paragraph 2.4.1) and the parallel design (paragraph 2.4.1), but is also more costly regarding the execution time [35, p. 7].

The Reflexive Self-Correcting Loop [35, p. 5] executes the Agents in a predefined order similar to the Sequential Pipeline (paragraph 2.4.1), but adds a verification step. The verification is performed by an Agent, which either deems the result to be correct (enough) and declares it to be the system's output or sends it back for correction. Before the result is reintroduced into the system, it is critiqued by stating the problems encountered, which allows for targeted corrections. This architecture refines the output over time rather than simply rerunning the entire system. The Reflexive Self-Correction Loop [35, p. 5] achieves the highest accuracy but also comes at the highest cost of all architectures discussed [35, p. 7].

When interpreting the performances of the architectures discussed, it should be taken into account that Kulkarni and Kulkarni [35, p. 14-15] tested them on their extraction capabilities from financial documents. Therefore, the performances could differ in different domains and for different tasks. When designing Agentic AI systems, the combination of multiple architectures is possible.

2.4.2 Communication Inside of Agentic AI Systems

Communication protocols are used to define how the different components of an Agentic AI system interact. An orchestration framework may implement some or all of these protocols or similar ones. The LLMs are wrapped into a layered architecture, which handles the interactions between different LLMs and manages how tools are called [23].

The Model Context Protocol (MCP) [20] (documentation: [45]) handles how the underlying LLM of an Agent uses tools.

The Agent Communication Protocol (ACP) [20] (documentation: [18]) allows different Agents to communicate with each other. It is based on a client-server architecture and uses "RESTful" communication [24].

The Agent-to-Agent Protocol (A2A) [20] (documentation: [43]) handles Agent interaction to enable effective cooperation. This is achieved by Agents communicating their capabilities [24] to each other.

The Agent Network Protocol (ANP) [20] (documentation: [6]) manages decentralised communication between Agents. It provides secure interactions over the internet [24].

2.4.3 Memory

In the context of Agentic AI, memory is an Agent's awareness of previous actions. Short-term memory is the context of the task the Agent is currently performing. Long-term memory allows an Agent to access information it obtained during previous runs [20]. Other kinds of memory include "semantic memory" [20], "procedural memory" [20], and "episodic memory" [20], but these will not be discussed in this Thesis.

2.5 Evaluation of AI Systems

In order to measure the influence of an AI system's configurations, a metric for the systems performance is required. One approach is testing the AI system on a set of multiple choice questions, with each answer being clearly right or wrong [25]. The total score over the set of questions indicates the system's performance. This method is used for the "GPQA Diamond" benchmark [40] and partly for the " Massive Multi-discipline Multimodal Understanding and Reasoning (MMMU)" [51] benchmark, which are both part of the Artifical Analysis *LLM Leaderboard* [2].

Another option are open answer questions, which require a written response by the tested model. This method is used by "SciBench"[48], where the answers usually are short and need to match the expected answer exactly, since a string comparison of them is performed. The performance over all questions is accumulated and provides a final evaluation score of the tested system.

In order to properly assess an AI system's capability to handle more complex tasks, human grading can be employed. This is done by Arena Intelligence [1], where users write a prompt and pick the best response from two anonymous answers. Human grading is also used by in "SciEx" [21], where experts evaluate the LLM's results. As acknowledged by Dinh et al., human grading is not feasible for large-scale benchmarking due to its time and resource inefficiency.

2.5.1 CritPt

Presented by Zhu et al., "CritPt" [53] introduces checkpoints which allow one to identify a model's progress on a question. This provides a more in-depth view of the failure points of the system under evaluation. The probed system is prompted with the question and generates an answer; afterwards, the required output format

must be adopted to enable automatic evaluation. Zhu et al. describe using the checkpoints to guide the system towards the correct answer and also mention a mechanism which assists the tested system in adopting the required output format.

There are three types of questions, which require either a numerical value, a symbolic expression, or executable python code. Composite answers are only graded to be correct if all individual parts are correct [53].

CritPt limits the number of requests made to the evaluation servers to 10 submissions per 24 hours per use,r in order to prevent leakage of the correct answers [53, p. 10]

3 Design of the Physics Modelling Assistant

In order to easily enable future work on and with the Physics Modelling Assistant, the choice was made to only use services, which are open source and free of charge.

3.1 Orchestration Framework: CrewAI

The decision was made not to design a new framework for orchestrating AIs in order to save time on this software development heavy task and instead focus on the Physic Modelling Assistant's configurations. Derouiche, Brahmi, and Mazeni [20] discuss a number of frameworks varying in complexity and beginner friendliness. For this Thesis, the CrewAI [15] framework is used for its role focused style [20], consisting of Agents and Tasks [15]. Every Agent can be configured individually, which includes using different LLMs for every Agent. Every Task has its own description and specification, including the Agent assigned to it. On execution of a Task, its definition is merged with the assigned Agent, providing the input for the LLM used. An Agent's behaviour is defined by its role, goal, and backstory. The role is a very short description comparable to a job title. The goal specifies the actions an Agent is supposed to perform similar to a job description. The backstory provides details on the Agent's behaviour regarding preferences or advice on how to approach a Task. CrewAI allows to access any LLM provided the user presents a valid API key. The LLM can be assigned parameters including the discussed Temperature(subsubsection 2.2.3) and top_p (subsubsection 2.2.5), the number of tokens to be used, and others. Every Task has a description and specifications regarding the output and the output format, along with the designated Agent designated to it [16]. Tools can be assigned to both Agents and Tasks, with CrewAI providing ready to use tools [11], but also allows to add custom tools.

The organisation of multiple CrewAI Agents and Tasks is managed by the Crew. It is responsible for validating the Agent's and Task's configurations beforehand and then execute the system, as seen in Figure 1. As stated by Derouiche, Brahmi, and Mazeni, [20] CrewAI implements an orchestration combining elements from the ACP, A2A and ANP (see subsubsection 2.4.2). The CrewAI documentation [15] and the source code available on GitHub [14] show a sequential and a hierarchical execution approach. The sequential execution (see Figure 1) runs the Tasks in the order they are defined in the source code, thereby forming a sequential Pipeline (see paragraph 2.4.1). This can be modified into a Parallel Fan-Out with Merge (see paragraph 2.4.1) by, allowing certain Tasks to run at the asynchronously [13]. Merging is handled by specific Task, waiting for all parallel Tasks to be done before starting its work. The hierarchical execution (see Figure 1) style of a CrewAI Crew enables a Hierarchical Supervisor-Worker architecture (see paragraph 2.4.1) by delegating Tasks, or aspects of them, to subtasks. Implementing a Reflexive Self-Correcting Loop (see paragraph 2.4.1)

with CrewAI can be achieved by making use of Flows [12], which allow for conditional calls and the coupling of multiple crews.

In order to achieve a deterministic workflow the Sequential Pipeline architecture (paragraph 2.4.1) is used for the Physics Modelling Assistant.

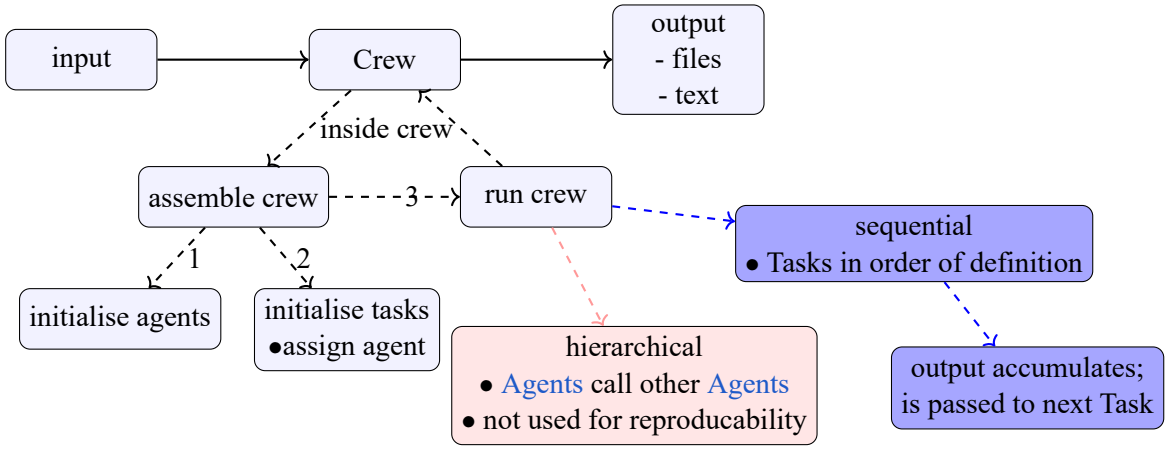

Figure 1: The main steps executed during a run of a CrewAI Crew. The Crew receives an input containing various parameters specifying the Crew's behaviour. This includes a message stating what the Crew should work an, as well as parameters shaping the LLM's behaviour (Temperature subsubsection 2.2.3, top_p subsubsection 2.2.5), and the Agent's behaviour (planning before execution, how many times to rerun itself [8]). The Crew is assembled by initializing all of its Agents and Tasks, before mapping the Agents to their corresponding Tasks. If there is a Task without an Agent, the Crew produces an error and does not proceed with the execution. After the Crew is assembled correctly, it can run either in a hierarchical or a sequential style. For the hierarchical execution one main Agent calls other (Sub-)Agents, which can call further (Sub-)Agents themselves, to aid them in accomplishing the given Task (see subsection 3.1). During the sequential execution the Agents, respectively the Tasks with their Agents, are executed in the order they are defined in the source code (see subsection 3.1).

3.2 LLMs Used

In order to use LLMs effectively in an automated pipeline a, provider offering a program access to its LLM is needed. For this reason the "Scalable AI Accellarator (SAIA)" [22] is accessed via the servers from the "Gesellschaft für wissenschaftliche Datenverarbeitung mbH Göttingen" (GWDG) [29]. The LLMs offered by SAIA can be accessed via an API key obtained from the GWDG. A list of the available LLMs is provided on the SAIA webpage [19], but can also be obtained via a request to the server hosting the models. To identify to most capable of the available models, individual benchmarks contributing to the *LLM Leaderboard* from Artifical Analysis are used [2]. The AA-LCR tests the long context reasoning, the GPQA Diamond tests for scientific reasoning capabilities, SciCode evaluates the models capabilities in science and coding, CritPt (see subsubsection 2.5.1) probes the physics based reasoning abilities of the LLMs. The performances of all models are displayed in Table 1 conclude qwen3.6 27B (27 billion parameters) to be the best model for long context reasoning, while qwen3.5 397B A17B (397 billion parameters in total, 17 billion active at a time) performs best in scientific reasoning. glm-4.7 (358 billion parameters [28]) achieves the best result in science and coding. For the CritPt benchmark both glm-4.7 and qwen3.5 397B A17B achieve the best score among the tested models, with both of them scoring at only 1.7 %, indicating the difficulty of the CritPt

benchmarking questions.

Table 1: Benchmarking results for the LLMs provided by SAIA [22]. The benchmarking results are taken from Artificial Analysis [3]. The chosen benchmarks are: AA-LCR evaluating long context reasoning, GPQA Diamnond evaluating scientific reasoning, SciCode evaluating coding and science capabilities, CritPt evaluating physics reasoning

model	AA-LCR in %	GPQA Diamond in %	SciCode in %	CritPt in %
apertus 70B Instruct	0	27	6	0
deepSeek V4 Flash	33	72	37	0
devstral 2	30	53	29	0
gemma 4 31B	62	86	43	1
glm-4.7	64	86	45	1.7
llama 3.1 8B	16	26	13	0
mistral Medium 3.5	61	75	40	0
gpt-oss-120B (high)	51	78	39	1
qwen3 30B A3B 2507	59	71	33	0
qwen3 coder next	40	74	32	0
qwen3 omni 30B A3B	0	73	31	0
qwen3.5 122B A10B	67	86	42	1
qwen3.5 397B A17B	66	89	42	1.7
qwen3.6 27B	69	84	40	1
qwen3.6 35B A3B	64	84	36	0

3.3 Tools Used

To enhance the Physics Modelling Assistant's capabilities, certain Agents are equipped with tools (see subsubsection 2.3.2). Each tool provides an object specifying the expected input at its constraints, both are visible to the Agent using the tool.

3.3.1 arxiv_paper_tool with Exponential Backoff

CrewAI already provides the "arxiv_Paper_Tool" [10], accessing the arXiv-API [4] for finding papers relevant to the question at hand. During the development process of the Physics Modelling Assistant, the CrewAI tool's requests repeatedly kept getting denied, resulting no papers to be found and downloaded from arXiv. As listed in the arXiv-API's Terms of Use, rate limits are enforced, blocking clients from sending many requests in a short amount of time. Furthermore, web servers will deny repeated request from the same client from the same service [49]. Since the CrewAI arxiv_paper_tool [10] does not implement any strategy for waiting, its requests are eventually blocked by the arXiv-API servers. As a consequence an extending to the arxiv_paper_tool was made by implementing an exponential back-off strategy, as it is common for modern server requests [49]. Additionally to the exponential back-off, the modified tool, called arxiv_paper_tool_exponential_backoff, adds a small random wait, which is common to prevent two clients requesting the same service from waiting for the same amount of time and blocking each others requests repeatability [49]. The tool finds a specified number of papers and downloads them to a designated directory.

3.3.2 pdf_reader

In order to access the papers downloaded from arXiv, a PDF reader tool was developed. It reads all PDF files the specified directory, allowing the invoking Agent's LLM to see the contents of the documents. In theory this allows to read any number and length of sources from the directory. In reality every LLM has a context-window, limiting the amount of tokens it can handle. When the sum of the tokens of the read documents exceeds the context-window, the Agent will fail its Task. Among the SAIA models [19], deepSeek V4 Flash offers the largest context-window, supporting up to 1 million tokens, while qwen3.6 35B A3B provides the second-largest context-window at 262k tokens.

Since the tool only allows reading files from a hard-coded subdirectory, it is deemed secure to be used outside of a sandbox environment (see subsubsection 2.3.2).

3.3.3 dimensional_analysis

During the testing of the system, formulas from source papers where stated correctly, but assigning correct units to the individual quantities proved to be difficult for the responsible Agent. Mistakes in the unit assumptions lead to unphysical results or impossible units for the outcomes of the numerical implementation. To verify the consistency for all formulas used, the dimensional_analysis tool was created. It is based on the symPy library and allows an Agent to check the units it proposes. The tool takes an equation written as a string, a dictionary containing the used quantities and their proposed units, as well as a list of all units that are used. The units are substituted in for their respective quantities and both sides of the formula are simplified separately. Numerical values are not considered, since the aim is simply to achieve unit consistency. Lastly, the left side of the formula is divide by the right side, giving the unit correction needed on the right side for the formula to be consistent. Ideally, this value will be 1, indicating no correction is needed. If it is not 1, the Agent is supposed to interpret the unit mismatch correctly, by adjusting the assumed units for the used quantities. Another option for the Agent is to modify the formula, which can be done by inserting physical constants. This allows to keep real world units while implementing a formula, which uses a different unit system.

3.3.4 Code Execution

As discussed by Marchand et al. [39], designing a secure Sandbox is a complex task but necessary if LLM generated code should be executed directly. As stated in subsubsection 2.3.2, the weak points of Sandboxes can be exploited by capable LLMs, which is why the Physics Modelling Assistant does not allow for code execution.

3.4 Experimental Setup for the Pipeline

The final system is designed to mirror a simplified research workflow consisting of the finding of relevant information, developing a model based on the knowledge acquired and implementing the model to get an idea of what it predicts. In order to give definitive answers to the questions from benchmarks (see subsection 2.5), a Task providing a comprehensive answer is run in the end.

A simplified view of the steps the Physics Modelling Assistant performs is shown in Figure 2, each step will be performed by a Task and its Agent.

3.4.1 Input

The Physics Modelling Assistant receives a single message (a string called **aim**) as its input. Additional parameters can be set via its interface. The output directory for the files created by the tasks can and should be specified to previous outputs from being overwritten. The Temperature (see subsubsection 2.2.3) and the top_p (see subsubsection 2.2.5) parameters can be specified to shape the LLM's output. The **max_tokens** parameter is set and will be used to control how many tokens a LLM produces. This is done to prevent the context-window to be exceeded, causing the Agent to fail its Task.

Each Agent can perform **reasoning** before it performs its Task. Reasoning causes the Agent to create a plan on how to approach the Task before working on it [9]. The maximum number of reasoning tries can be specified, otherwise reasoning will be retried until its results are deemed sufficient [9]. An Agent can also rerun itself to improve its output if it deems this to be necessary, the maximum amount of iterations is expressed through the **max_iter** parameter [9].

The directory for the Tasks output files and the PDF downloads can be specified in the input as well.

3.4.2 General Parameter Choices

Most parameters are to the same for all Agent. This is not done arbitrarily but based on the recommendation from the SAIA information [19]. For glm-4.7 and qwen3.6 27B they are: **Temperature** = 1.0 and **top_p** = 0.95. For qwen3.5 397B the recommendation is **Temperature** = 0.6 and **top_p** = 0.95 but this model will not be used for the reasons discussed in subsubsection 3.4.4.

The maximum number of output tokens is set to 50k [9] to ensure the context window is not exceeded. This is especially relevant for the Agent extracting information from the papers.

Each Agent is allowed up to three iterations (**max_iter** = 3) to refine its result if it deems it necessary [9]. This allows a single Agent's try to be improved if its output is of bad quality due to the complexity of the Task or the inherent randomness of LLMs. Furthermore, infrequent failures like context-window exceedances do not cause an Agent to fail immediately.

Reasoning allows Agents to make a step by step plan on how to approach a Task before executing it [9]. The **max_reasoning_attempts** specify the upper bound for the attempts for reasoning. It is generally stet to 3 for all Agents, otherwise reasoning would continues until the Agent is satisfied with the attempt, bearing the risk of many retries [9].

3.4.3 Obtaining Information

Soure Gathering In order to get sufficient information for building a model, the arXiv API [4] will be accessed via the arxiv_paper_tool (see subsubsection 3.3.1) and then downloaded. The corresponding **source**

gatherer Agent is based on the qwen3.6 27B LLM and is tasked to use its tool for finding papers related to the aim. The **gathhering task** specifies that three to six papers should be found, more papers bare the risk of exceeding the context-window of the used LLMs. The Task further specifies which information should be listed for each source: arXiv ID, author, data, URL and a short summary. This is then written into a markdown report allowing a human to inspect the sources found.

This Agent does not use reasoning because its Task is deemed to not need complex multi step approach. All other parameters are kept at the global settings as discussed in subsubsection 3.4.2.

Because of the issues with the arXiv API rate limits [4], this step is packaged into its own Crew, allowing it to be executed independently from all later steps. This way changes or reruns intended to result in different behaviour of later steps can be made without rerunning the arXiv request again, which is going to fetch similar (most often the same) papers for the same aim.

Information Extraction The source finding Task saves its papers to the specified output directory, by knowing the directory, the **information extractor** Agent will use its pdf_reader (see subsubsection 3.3.2) read the content of the files in that directory. The qwen3.6 27B LLM is used for its superior long context reasoning score (see Table 1). Its context-window of 262k tokens is the second largest to deepSeek V4 Flash's 1M [22] [19]. Since deepSeek V4 Flash achieved less than half of qwen3.6 27B's score in long context reasoning Table 1, the latter will be used.

The **information extractor** is instructed to find the information needed to build a model that accomplishes what the aim asks for. It should present its findings with scientific citation and its backstory emphasises that the Agent should stay true to the source material it has available. The corresponding **extraction task** specifies that the output should be in markdown format, enforcing an unambiguous and human readable output for later Agents and inspecting the Tasks markdown output file later.

The parameters are kept from the global settings as described in subsubsection 3.4.2

3.4.4 Building the Model

Modelling For building the model which is at the heart of the Physics Modelling Assistant, the LLM strongest in scientific and specifically physics reasoning should be used. Therefore, the first choice was qwen3.5 397B A17B performing the best at scientific reasoning and joined best at CritPt [53], as shown in Table 1. During the testing phase, requests to this model kept failing with an answer not being provided by the server before a timeout was reached. For that reason the glm-4.7 LLM was chosen instead, for it being the joined second best model at scientific reasoning and joined best model at CritPt [53] (see Table 1). The **modeller** Agent is prompted to provide a mathematical description for a model which performs the aim. It is further instructed to rely on the information provided by the previous Task (extraction task), provide a proper explanation for the steps taken and not to write any code. The last instruction targets the Agent's previous behaviour of occasionally writing code which already implements the model. This is not wanted due to potential modifications needed for the model to be sufficient.

The **modelling task** defines the expected output to be a complete mathematical description with a textual explanation. The requested format is once again markdown ensuring human readability of the generated output file. The default parameters setting from subsubsection 3.4.2 are adopted.

Unit Check The **unit checker** Agent accounts for custom parameter definitions and other conventions preventing a straight forward adoption of the model and its formulas. It uses the glm-4.7 LLM and its goal specifies which steps to take. In the beginning the units of all the quantities should be determined and based on these assumptions, the dimensional_analysis tool (see subsubsection 3.3.3) will be used to identify potential inconsistencies in the units assumed. Based on these results, the Agent is expected to correct its unit assumptions or formulas if deemed necessary.

The **unit checking task** specifies that the quantities of the units and the output of the dimensional analysis tool should be displayed in the markdown output file. The parameter configuration follows the default.

Parameters Suggestion In order to obtain a meaningful simulation, physically realistic starting conditions are needed. They will be provided by the **parameter suggester**, which has the goal of finding typical starting parameters for the model simulation and stating how these suggestions where derived. The **parameter suggestion task** highlights that explanations for the choice of parameters should be included in the markdown output file. The input parameters are unchanged compared to the default and the glm-4.7 model is used for the reasons discussed in paragraph 3.4.4.

3.4.5 Implementing the Model

Physics Simulation In order to capture the suggested model accurately, the **simulation physician** Agent is run with the goal of implementing the formulas of the model precisely without making changes to them. The Agent is based on the glm-4.7 LLM performing best in coding and science among the available models (see Table 1). As specified by the **physician simulation task**, the implementation is supposed to use the units determined by the unit checking task. To get human readable and interpretable results, the task is requested to create graphics if it seems sensible. The output will be a Python file which should comment all contents which are not valid code. The parameters are kept as before.

Simulation Correction The physician simulation task is meant to capture the models logic into working code and does not prioritize perfectly correct code according to its specifications. The correction of bugs is done by the **simulation correcter**, which should not change the implemented formulas but is only supposed to correct coding mistakes and improve the codes efficiency and style. The corresponding **simulation correction task** adds to the Agent definition by specifying the expected output to be working python code. The parameter configurations and LLM are unchanged compared to the simulation physician in paragraph 3.4.5.

3.4.6 Giving a Final Answer

Benchmarks like CritPt expect a comprehensive answer (see subsection 2.5) to their questions, for that reason the **final outpter** Agent is added to the pipeline. Its goal is to give a final answer including the derivation of it. The format requested by the question should be obeyed, which is necessary for grading as described for CritPt [53]. The **final output task** specifies the output is supposed to be a string and produces a markdown output file. The glm-4.7 model is used for its performance in the CritPt benchmark as seen in Table 1 and the issues with qwen3.5 397B A17B discussed in paragraph 3.4.4.

3.4.7 Output

The output of the Physics Modelling Assistant is an object from which the string containing the last Task's result is extracted as the answer for benchmarks questions. Output files generated by each Task can be inspected by a human, the Python scripts are not directly executable since they contain triple backticks. When commenting them the Python files become executable amid possible package installations.

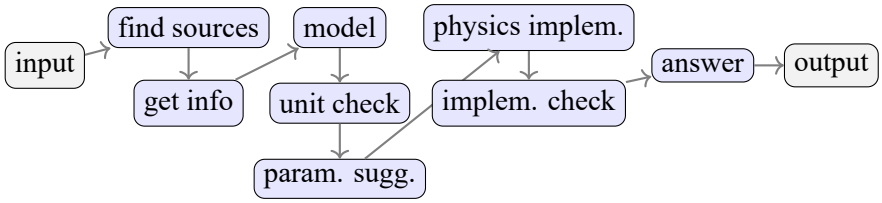

Figure 2: Overview of all the Tasks and their order as they are performed by the Physics Modelling Assistant. The system receives an input string as a free form modelling request; it will find papers on that topic and download them from arXiv; the papers will be read and necessary information gets extracted; a model is build based on the extracted information; the formulas of the model are checked for unit consistency; realistic starting parameters are suggested for the simulation; the model is implemented into Python code; the code is checked and corrected if necessary; a final answer to the input is provided; output is the final answer and the files created by the individual tasks

3.4.8 Connecting to CritPt's Benchmark

CritPt is based on the inspect_ai so for benchmarking, the Physics Modelling Assistant adapt to it. After installing CritPt according the installation instructions [17] [53], a new inspect_ai modelapi is registered outside of the CritPt directory. The registration calls the PMAModelAPI class implementing the inspect_ai ModelAPi class with a valid generate method. The question is extracted from the input received a the parameters necessary for the Physics Modelling assistant are set in the generate method. With the set parameters a subprocess is called, connection to the Physics Modelling Assistant. The use of a subprocess is necessary due to dependency conflicts of the crewai and inspect_ai package preventing them from using the same versions of the tiktoken and pydantic package. The subprocess allows to call the Physics Modelling Assistant from another virtual environment than CritPt is running in. By using the asyncio package, the super-process waits unit the completion of the subprocess. The output of the Physics Modelling Assistant is read from a file written by the subprocess. The file is generated by the manager of the subprocess, which calls the Physics Modelling Assistant and handles the technical issues of the communication.

The generated results are stored in json-files and can be send to the evaluation server via a script provided by CritPt. Since Artificial Analysis [3] has integrated CritPt, a valid API key is needed for the evaluation server. It can be obtained via Artificial Analysis and is free of charge.

No checkpoints, format needs to be adopted by system itself. Only final answer matters

3.5 Investigating the Influences on the Physics Modelling Assistant's Performance

3.6 Pipelines Performance Compared to Individual Models

The Physics Modelling Assistant's performance is measured and compared to the results of the individual LLM's performances stated in Table 1. The Pipelines performance in the described setup (see subsection 3.4) is used as a reference for other changes.

3.6.1 The Relevance of the Available Sources

The influence of the sources available to the Pipeline is measured.

The Pipeline runs with the deepSeek V4 Flash LLM for the information extraction (see paragraph 3.4.3) and its performance is compared to the reference using qwen3.6 27B.

Instead of papers from arXiv as used in the reference, a physics textbook is provided to the extraction task and the performance of this setup is compared to the reference. """"maybe too long for context-window""""

The Physic Modelling Assistant's performance is measured when it has no external sources available. The performance is compared to the the reference setup.

3.6.2 The Effect of Iterations of the Agents

All Agents of the Physics Modelling Assistant receive max_iter = 1, which means they can not reiterate. The result are compared with the setup using the default maximum value of 3 iterations.

3.6.3 The Effect of Reasoning

The Pipeline is run without reasoning, therefore not allowing the Agents to create a plan before execution a Task. This setup's performance is compared with the reference, which enables a maximum of 3 reasoning attempts per Agent .

4 Results and Discussion

4.1 The Physics Modelling Assistant's Performance in the Reference Setup

- Example of important steps for one problem

Performance

4.2 The Performance for Different Available Sources

Compare performance for all different

4.3 The Performance With and Without Reiteration

compare

4.4 The Performance With and Without Reasoning

compare

5 Conclusion and Future Work

- does the PMA perform better than its individual models: expecially interesting for the weaker models compared to physics intern - which settings influence performance the most

- future work: integrate true RAG DB, test if performance improve; systematically investigate the ideal settings for reasoning and max_iter: what is the threshold at which more steps do not improve the results; implement feedback loop into Pipeline

References

[1] Arena Intelligence 2026, ed. *Arena*. URL: https://arena.ai/how-it-works (visited on 07/29/2026) (cit. on p. 12).

[2] Artifical Analysis. *LLM Leaderboard. Comparison of AI models from OpenAI, Anthropic, Google, SpaceXAI & others*. URL: https://artificialanalysis.ai/leaderboards/models (visited on 07/30/2026) (cit. on pp. 12, 14).

[3] Artificial Analysis, ed. *Independent analysis of AI*. URL: https://artificialanalysis.ai/ (visited on 07/30/2026) (cit. on pp. 15, 20).

[4] arXiv. *arXiv API Access*. 07/09/2026. URL: https://info.arxiv.org/help/api/index.html (cit. on pp. 15, 17, 18).

[5] Ajay Bandi, Bhavani Kongari, Roshini Naguru, Sahitya Pasnoor, and Sri Vidya Vilipala. "The Rise of Agentic AI: A Review of Definitions, Frameworks, Architectures, Applications, Evaluation Metrics, and Challenges". In: *Future Internet* 17.9 (2025). ISSN: 1999-5903. DOI: 10.3390/fi17090404. URL: https://www.mdpi.com/1999-5903/17/9/404 (cit. on p. 6).

[6] Gaowei Chang. *Agent Network Protocol*. URL: https://agent-network-protocol.com/ (visited on 07/29/2026) (cit. on p. 12).

[7] Marina Chugunova, Dietmar Harhoff, Katharina Hölzle, Verena Kaschub, Sonal Malagimani, Ulrike Morgalla, and Robert Rose. "Who uses AI in research, and for what? Large-scale survey evidence from Germany". In: *Research Policy* 55.2 (2026), p. 105381. ISSN: 0048-7333. DOI: https://doi.org/10.1016/j.respol.2025.105381. URL: https://www.sciencedirect.com/science/article/pii/S0048733325002100 (cit. on p. 5).

[8] CrewAI. *Agent/core*. URL: https://github.com/crewAIInc/crewAI/blob/main/lib/crewai/src/crewai/agent/core.py (visited on 07/30/2026) (cit. on p. 14).

[9] CrewAI. *Agents*. URL: https://docs.crewai.com/v1.15.11/en/concepts/agents (visited on 08/05/2026) (cit. on p. 17).

[10] CrewAI. *ArxivPaperTool*. URL: https://github.com/crewAIInc/crewAI/tree/main/lib/crewai-tools/src/crewai_tools/tools/arxiv_paper_tool (visited on 07/30/2026) (cit. on p. 15).

[11] CrewAI. *crewai tools*. URL: https://github.com/crewAIInc/crewAI/tree/main/lib/crewai-tools/src/crewai_tools (visited on 07/30/2026) (cit. on p. 13).

[12] CrewAI. *Flows*. URL: https://docs.crewai.com/v1.15.9/en/concepts/flows (visited on 07/30/2026) (cit. on p. 14).

[13] CrewAI. *Tasks*. URL: https://docs.crewai.com/v1.15.9/en/concepts/tasks#asynchronous-execution (visited on 07/30/2026) (cit. on p. 13).

[14] crewai. *crew.py*. 04/22/2026. URL: https://github.com/crewAIInc/crewAI/blob/main/lib/crewai/src/crewai/crew.py (visited on 04/22/2026) (cit. on p. 13).

[15] crewai. *crewAi*. 04/22/2026. URL: https://crewai.com/ (visited on 04/22/2026) (cit. on p. 13).

[16] crewai. *task.py*. 04/22/2026. URL: https://github.com/crewAIInc/crewAI/blob/main/lib/crewai/src/crewai/task.py (visited on 04/22/2026) (cit. on p. 13).

[17] critPt. *challanges*. 04/22/2026. URL: https://github.com/CritPt-Benchmark/CritPt/tree/main/data/public_test_challenges (visited on 04/22/2026) (cit. on p. 20).

[18] Linux Foundation AI & Data. *Agent Communication Protocol*. URL: https://agentcommunicationprotocol.dev/about/mission-and-team (visited on 07/29/2026) (cit. on p. 12).

[19] Gesellschaft für wissenschafliche Datenverarbeitung mbH Göttingen, ed. *SAIA*. URL: https://docs.hpc.gwdg.de/services/ai-services/saia/index.html#api-request (visited on 07/30/2026) (cit. on pp. 14, 16–18).

[20] Hana Derouiche, Zaki Brahmi, and Haithem Mazeni. *Agentic AI Frameworks: Architectures, Protocols, and Design Challenges*. 2025. arXiv: 2508.10146 [cs.AI]. URL: https://arxiv.org/abs/2508.10146 (visited on 07/28/2026) (cit. on pp. 11–13).

[21] Tu Anh Dinh, Carlos Mullov, Leonard Bärmann, et al. *SciEx: Benchmarking Large Language Models on Scientific Exams with Human Expert Grading and Automatic Grading*. 2024. arXiv: 2406.10421 [cs.CL]. URL: https://arxiv.org/abs/2406.10421 (cit. on p. 12).

[22] Ali Doosthosseini, Jonathan Decker, Hendrik Nolte, and Julian Kunkel. "SAIA: a seamless Slurm-native solution for HPC-based services". In: *The Journal of Supercomputing* 82.7 (05/2026), p. 403. ISSN: 1573-0484. DOI: 10.1007/s11227-026-08508-3. URL: https://doi.org/10.1007/s11227-026-08508-3 (cit. on pp. 14, 15, 18).

[23] Yogesh K. Dwivedi, Mohamed Y. I. Helal, Ibrahim A. Elgendy, Rasha Alahmad, Paul Walton, Ayoung Suh, Vinay Singh, and Il Jeon. "Agentic AI Systems: What It Is and Isn't". In: *Global Business and Organizational Excellence* 45.3 (2026), pp. 253–263. DOI: https://doi.org/10.1002/joe.70018. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/joe.70018. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/joe.70018 (visited on 07/28/2026) (cit. on p. 11).

[24] Abul Ehtesham, Aditi Singh, Gaurav Kumar Gupta, and Saket Kumar. *A survey of agent interoperability protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP)*. 2025. arXiv: 2505.02279 [cs.AI]. URL: https://arxiv.org/abs/2505.02279 (visited on 07/29/2026) (cit. on p. 12).

[25] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. *Measuring Massive Multitask Language Understanding*. 2021. arXiv: 2009.03300 [cs.CY]. URL: https://arxiv.org/abs/2009.03300 (visited on 07/29/2026) (cit. on p. 12).

[26] HuggingFace. *Agent Arena*. URL: https://huggingface.co/spaces/lmarena-ai/arena-leaderboard (visited on 07/28/2026) (cit. on p. 9).

[27] HuggingFace. *Artificial Analysis LLM Leaderboard*. URL: https://huggingface.co/spaces/ArtificialAnalysis/LLM-Performance-Leaderboard (visited on 07/28/2026) (cit. on p. 9).

[28] HuggingFace. *GLM-4.7-FP8*. URL: https://huggingface.co/zai-org/GLM-4.7-FP8 (visited on 07/30/2026) (cit. on p. 14).

[29] *Imprint*. URL: https://gwdg.de/imprint/ (visited on 07/30/2026) (cit. on p. 14).

[30] Jens Jacobi. "Ein Pascal-orientiertes Unterstützungssystem für grafische Darstellungen und Verarbeitung von Meßdaten in der Biotechnologie". Diplomarbeit. Ingeneurshochschule Köthen
Sektion Biotechnologie, Lebensmitteltechnologie
Sektion Mathematik, Informatik, Rechentechnik, 01/1990 (cit. on p. 6).

[31] Xiaotong Ji, Rasul Tutunov, Matthieu Zimmer, and Haitham Bou-Ammar. *Decoding as Optimisation on the Probability Simplex: From Top-K to Top-P (Nucleus) to Best-of-K Samplers*. 2026. arXiv: 2602.18292 [cs.LG]. URL: https://arxiv.org/abs/2602.18292 (visited on 07/27/2026) (cit. on pp. 4, 7–9).

[32] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. *Scaling Laws for Neural Language Models*. 2020. arXiv: 2001.08361 [cs.LG]. URL: https://arxiv.org/abs/2001.08361 (visited on 07/28/2026) (cit. on pp. 9, 10).

[33] Mojtaba Komeili, Kurt Shuster, and Jason Weston. *Internet-Augmented Dialogue Generation*. Ed. by Smaranda Muresan, Preslav Nakov, and Aline Villavicencio. Dublin, Ireland, 05/2022. DOI: 10.18653/v1/2022.acl-long.579. URL: https://aclanthology.org/2022.acl-long.579/ (visited on 07/28/2026) (cit. on p. 10).

[34] Vikram Krishnamurthy. *LLMs as High-Dimensional Nonlinear Autoregressive Models with Attention: Training, Alignment and Inference*. 2026. arXiv: 2602.00426 [cs.LG]. URL: https://arxiv.org/abs/2602.00426 (cit. on p. 6).

[35] Siddhant Kulkarni and Yukta Kulkarni. *Benchmarking Multi-Agent LLM Architectures for Financial Document Processing: A Comparative Study of Orchestration Patterns, Cost-Accuracy Tradeoffs and Production Scaling Strategies*. 2026. arXiv: 2603.22651 [cs.AI]. URL: https://arxiv.org/abs/2603.22651 (visited on 07/28/2026) (cit. on pp. 10, 11).

[36] Emmy Liu, Amanda Bertsch, Lintang Sutawika, et al. *Not-Just-Scaling Laws: Towards a Better Understanding of the Downstream Impact of Language Model Design Decisions*. 2026. arXiv: 2503.03862 [cs.CL]. URL: https://arxiv.org/abs/2503.03862 (visited on 07/28/2026) (cit. on p. 10).

[37] David Louapre, Joel Niklaus, and Lewis Tunstall. "physics-intern: an autonomous agentic framework for physics research". In: *Hugging Face* (04/12/2026). URL: https://huggingface.co/spaces/huggingface/physics-intern (visited on 07/24/2026) (cit. on p. 5).

[38] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. *The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery*. 2024. arXiv: 2408.06292 [cs.AI]. URL: https://arxiv.org/abs/2408.06292 (cit. on p. 5).

[39] Rahul Marchand, Art O Cathain, Jerome Wynne, Philippos Maximos Giavridis, Sam Deverett, John Wilkinson, Jason Gwartz, and Harry Coppock. *Quantifying Frontier LLM Capabilities for Container Sandbox Escape*. 2026. arXiv: 2603.02277 [cs.CR]. URL: https://arxiv.org/abs/2603.02277 (visited on 07/30/2026) (cit. on pp. 10, 16).

[40] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. *GPQA: A Graduate-Level Google-Proof Q&A Benchmark*. 2023. arXiv: 2311.12022 [cs.AI]. URL: https://arxiv.org/abs/2311.12022 (visited on 07/29/2026) (cit. on p. 12).

[41] Stuart Russell and Peter Norvig. *Künstliche Intelligenz Ein moderner Ansatz*. Pearson Benelux B.V. (Zweigniederlassung Deutschland), 2023. ISBN: 9783868944303. URL: https://elibrary.pearson.de/book/99.150005/9783863263263 (visited on 07/26/2026) (cit. on p. 6).

[42] Erzhuo Shao, Yifang Wang, Yifan Qian, Zhenyu Pan, Han Liu, and Dashun Wang. "SciSciGPT: advancing human–AI collaboration in the science of science". In: *Nature Computational Science* (03/01/2026). URL: https://doi.org/10.1038/s43588-025-00906-6 (visited on 07/24/2026) (cit. on p. 5).

[43] Rao Surapaneni, Miku Jha, Michael Vakoc, and Todd Segal. *Announcing the Agent2Agent Protocol (A2A)*. 04/09/2025. URL: https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/ (visited on 07/29/2026) (cit. on p. 12).

[44] InternAgent Team, Bo Zhang, Shiyang Feng, et al. *InternAgent: When Agent Becomes the Scientist – Building Closed-Loop System from Hypothesis to Verification*. 2025. arXiv: 2505.16938 [cs.AI]. URL: https://arxiv.org/abs/2505.16938 (cit. on p. 5).

[45] LCC) The Linux Foundation Project (LF Projects. *Model Context Protocol*. URL: https://github.com/modelcontextprotocol (visited on 07/29/2026) (cit. on p. 11).

[46] Romal Thoppilan, Daniel De Freitas, Jamie Hall, et al. *LaMDA: Language Models for Dialog Applications*. 2022. arXiv: 2201.08239 [cs.CL]. URL: https://arxiv.org/abs/2201.08239 (visited on 07/28/2026) (cit. on p. 10).

[47] Hanchen Wang, Tianfan Fu, Yuanqi Du, et al. "Scientific discovery in the age of artificial intelligence". In: *Nature* (08/01/2023). URL: https://doi.org/10.1038/s41586-023-06221-2 (visited on 07/24/2026) (cit. on p. 5).

[48] Xiaoxuan Wang, Ziniu Hu, Pan Lu, Yanqiao Zhu, Jieyu Zhang, Satyen Subramaniam, Arjun R. Loomba, Shichang Zhang, Yizhou Sun, and Wei Wang. *SciBench: Evaluating College-Level Scientific Problem-Solving Abilities of Large Language Models*. 2024. arXiv: 2307.10635 [cs.CL]. URL: https://arxiv.org/abs/2307.10635 (visited on 07/29/2026) (cit. on p. 12).

[49] Sandro Wefel. *Rechnernetze und verteilte Systeme WiSe 2025/2026*. URL: https://mos.uni-halle.de/Download/Aktuell/GB/MH_13108_aktuell.pdf (visited on 07/30/2026) (cit. on p. 15).

[50] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. *The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search*. 2025. arXiv: 2504.08066 [cs.AI]. URL: https://arxiv.org/abs/2504.08066 (cit. on p. 5).

[51] Xiang Yue, Yuansheng Ni, Kai Zhang, et al. *MMMU: A Massive Multi-discipline Multimodal Understanding and Reasoning Benchmark for Expert AGI*. 2024. arXiv: 2311.16502 [cs.CL]. URL: https://arxiv.org/abs/2311.16502 (visited on 07/29/2026) (cit. on p. 12).

[52] Wayne Xin Zhao, Kun Zhou, Junyi Li, et al. *A Survey of Large Language Models*. 2026. arXiv: 2303.18223 [cs.CL]. URL: https://arxiv.org/abs/2303.18223 (cit. on p. 6).

[53] Minhui Zhu, Minyang Tian, Xiaocheng Yang, et al. *Probing the Critical Point (CritPt) of AI Reasoning: a Frontier Physics Research Benchmark*. 2025. arXiv: 2509.26574 [cs.AI]. URL: https://arxiv.org/abs/2509.26574 (cit. on pp. 12, 13, 18–20).

6 Appendix

7 Declaration of authorship

I hereby declare that I have written this thesis independently and without unauthorized assistance.

All sources and references used have been properly cited.

This thesis has not been submitted in the same or substantially similar form for any other degree or academic qualification.

Halle (Saale), 24th August 2026

Janis Felix Jacobi

8 Acknowledgement

The authors gratefully acknowledge the computing time granted by the KISSKI project. The calculations for this research were conducted with computing resources under the project 9e28b773-a710-4a17-a242-523e92c812de.